

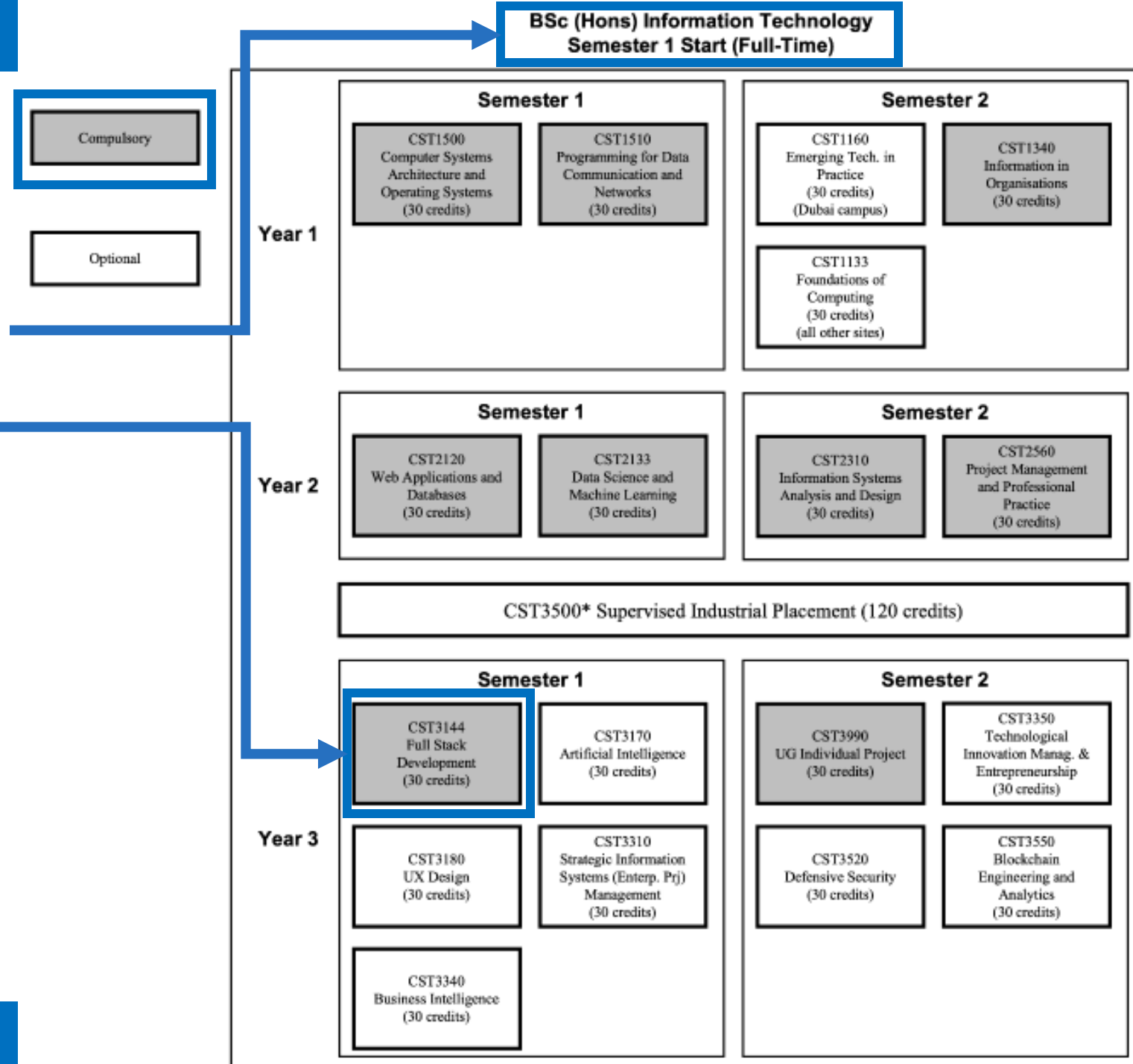
CST3144
Full Stack
Development

Module Intro

JavaScript
vs
Vue.js

Who am I?

- Dr Luca Piras
- Programme Leader of BSc IT
- Module Leader of **CST3144** Full Stack Development



Intro: Icebreakers

- Info about you
- Info about me
- Quick Icebreakers



Join at:
vevox.app

ID:
176-474-166



The Importance on Sharing your Journey with New Friends

- **Support Each other** Academically
- **Help** Navigate University Life
- Build **Lifelong Friendships**
- **Networking** of the future
- **Enjoy** the Journey Together
- In **this Module** we will a good balance of:
 - **Individual Work**
 - **Groupwork**
- **Icebreakers Activity:**
 - **speak** with your **colleague** and **share** some interesting **info** about **you**



Our Sessions

Workshop: 2 hrs (activities including for example Theory, Practice, Tutorials, CW Demo, CW Support, CW Q&A, etc., depending on the stage of the Module)

- **55 minutes** Workshop **Part 1**
- **10 minutes Break**
- **55 minutes** Workshop **Part 2**

Lab: 3 hrs

Drop-in online session: 1 hr

Outline and Learning Objectives

- **Background on App Development and Info on Lectures and Books:**
 - to understand how at high-level different approaches for App Development
 - to understand how we will approach to the Module and what Books to use
- **Integrated Developments Environments (IDEs):**
 - to understand what are IDEs and their main functionalities/support offered to the professional developer
 - to understand how to use an IDE, Visual Studio Code, in relation to Code Completion, Using Extensions and Local Servers
- **Comparison of JavaScript and Vue.js via an Initial Example:**
 - to understand how to develop a small example with JavaScript
 - to understand how to develop the same small example with Vue.js
 - to learn in relation to initial differences between JavaScript and Vue.js, by comparing the 2 small examples, and overview on a slightly more complex one
 - **Suggestions for Reading**

Intro: Full Stack Development

Module Overview Recap

- In this part we do a quick **Recap** of the “**Module Overview Lecture Recording**” on our “**Week 1a: Module Introduction and Guidance**” of our Module Page
- Then, today we will start covering the content of “**Week1b: JavaScript vs Vue.js**”

Week 1: Module Overview and JavaScript vs Vue.js

Week 1: Module Overview



Lecture-Module-Overview 969.3 KB PDF document



Module Overview Lecture Recording



Guidance video concerning: Module Page Overview and Blended Learning



Guidance video concerning: Reading List



Guidance video concerning: Handbook Overview



Guidance video concerning: Indicative Learning Planner (i.e. Schedule of our Module, week by week)



Guidance video concerning: Extensions / Deferrals and Extenuating Circumstances



Guidance video concerning: Late Penalty



Guidance video concerning: Incentive System

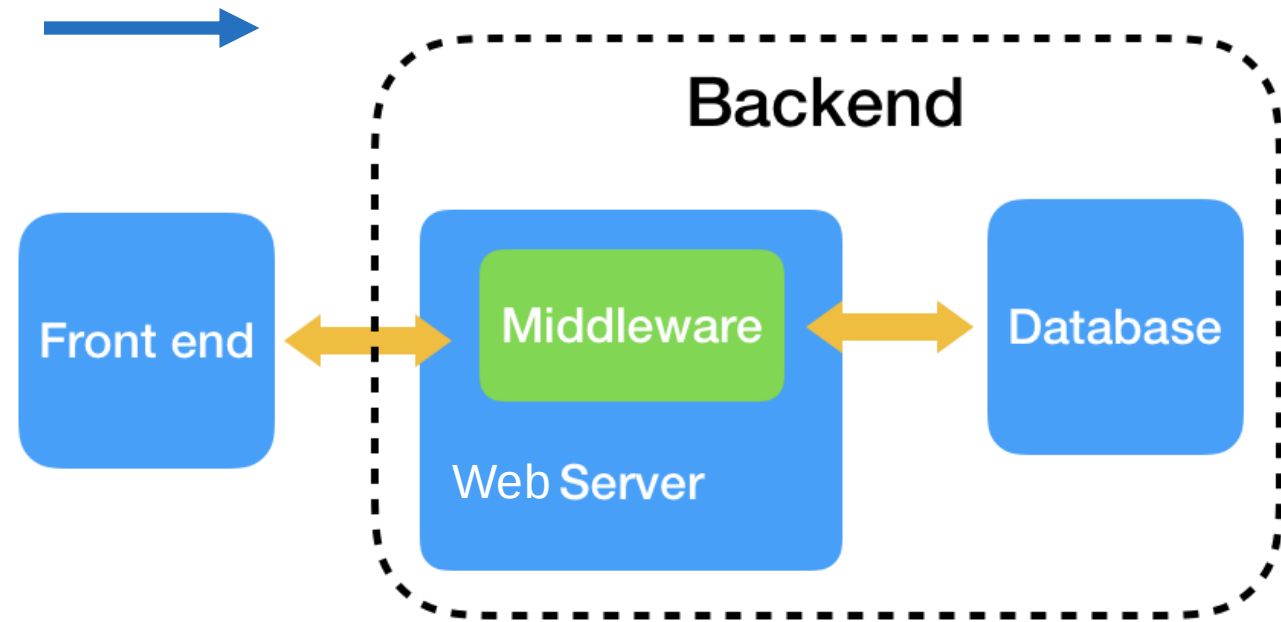


Guidance video concerning: Demonstration Process and Important Requirements/Instructions

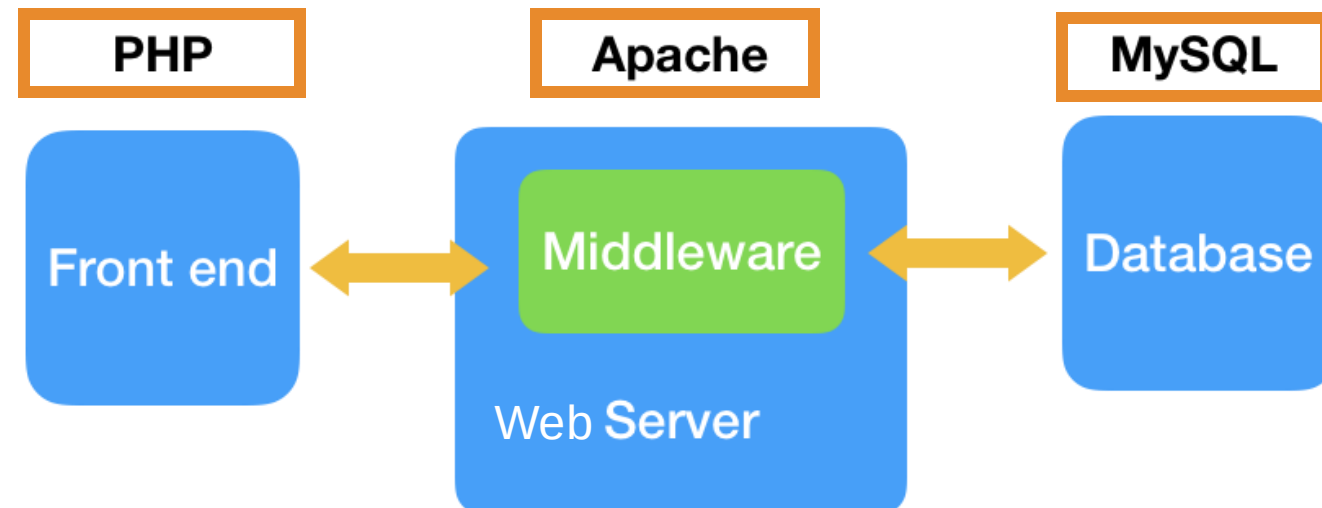
Week 1: JavaScript vs Vue.js

What is a Web Development Stack?

- All the components needed to run a Web app:
 - Front-End
 - Back-End: Web Server, Middleware, Database



- Popular stacks, for instance **xAMP**
 - **xAMP: x**, Apache, MySQL and **PHP**
 - **LAMP: Linux**, Apache, MySQL and **PHP**
 - **MAMP: MacOS**, Apache, MySQL and **PHP**
 - **WAMP: Windows**, Apache, MySQL and **PHP**



- Multiple languages: PHP, Java and SQL

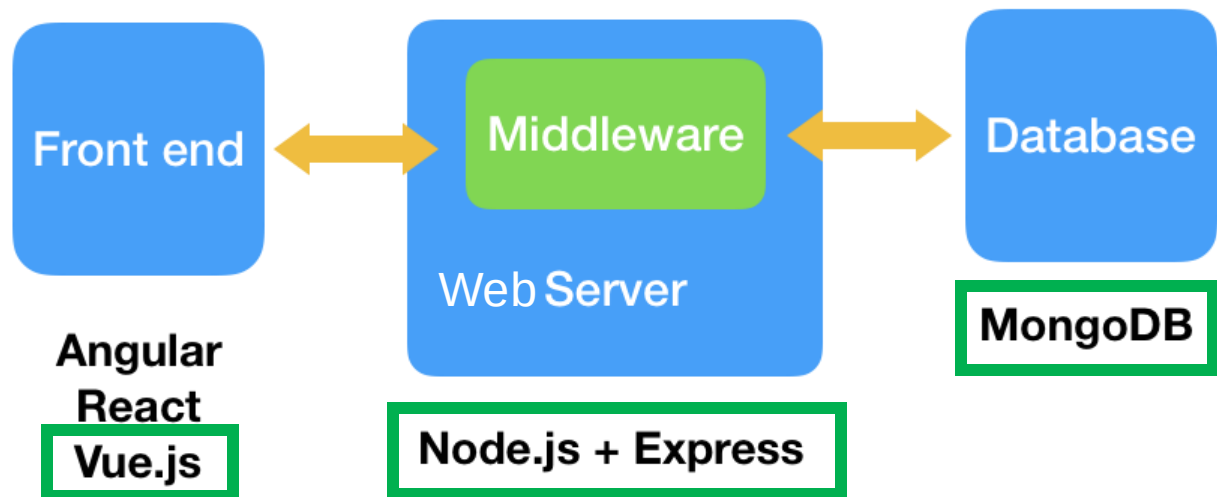
Popular Stacks MExN and Our Approach

Everything
will be
hosted
online

Other Popular stacks, MExN

- **MExN:** MongoDB, Express, **x** and Node.js
- **MEAN:** MongoDB, Express, **Angular** and Node.js
- **MERN:** MongoDB, Express, **React** and Node.js
- **MEVN:** MongoDB, Express, **Vue.js** and Node.js

- [Advantage] not using multiple languages,
 - only using JavaScript (MongoDB is “JavaScript-Friendly” by using JSON like structure)



Our Approach is Project-Based and:

- We will focus on **MEVN:** MongoDB, Express, Vue.js, and Node.js
- We cover Vue.js as first framework
- Then we cover Node.js (the server)
- Then Express.js (middleware), which is built on top of Node
- We will briefly cover MongoDB (database), as it is covered in the 2nd-year web dev module
- We will also cover how to host Node/Express (within Amazon Web Services, AWS) and MongoDB (Atlas) online.

Full-Fledged Application

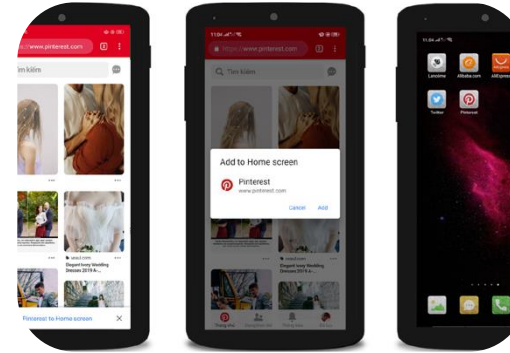
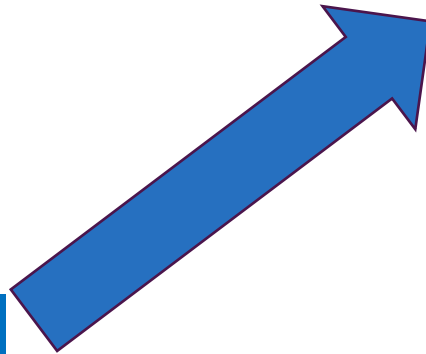
- Full Stack Development is:

- **mature** and **well-established**
- **widely used**

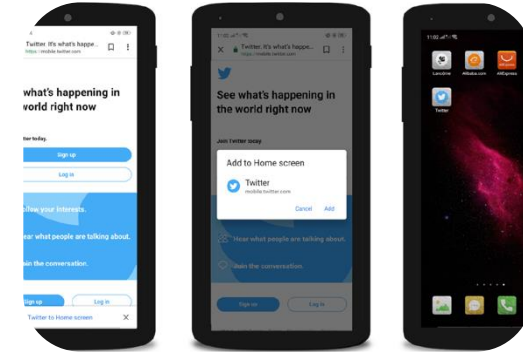
- Many of the **most used apps** follow such approach:

- Facebook
- Instagram
- AirBnB
- Uber
- Discord
- Skype

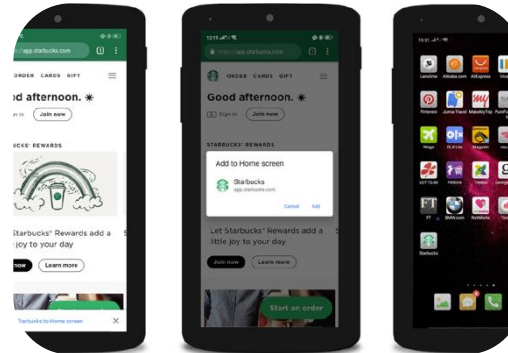
- Pinterest
- Twitter Lite
- Starbucks
- ...



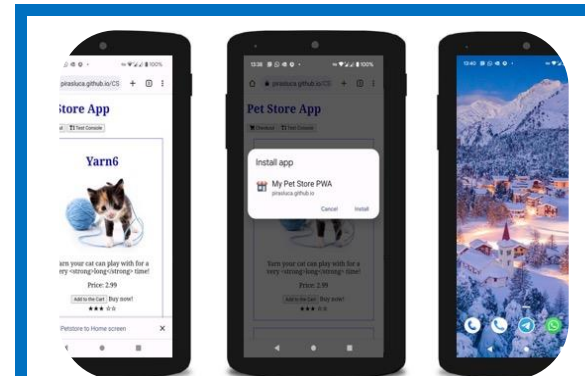
Pinterest



X (Previously Twitter)



Starbucks



Our Final App
(Project-Based Approach)

Books

-  Lecture-Module-Overview 969.3 KB PDF document
-  Module Overview Lecture Recording
-  Guidance video concerning: Module Page Overview and Blended Learning
-  Guidance video concerning: Reading List
-  Guidance video concerning: Handbook Overview
-  Guidance video concerning: Indicative Learning Planner (i.e. Schedule of our Module, week by week)
-  Guidance video concerning: Extensions / Deferrals and Extenuating Circumstances
-  Guidance video concerning: Late Penalty
-  Guidance video concerning: Incentive System
-  Guidance video concerning: Demonstration Process and Important Requirements/Instructions

Books and Reading List

- Full details at “**Guidance video concerning: Reading List**” on our “**Week 1a: Module Introduction and Guidance**” of our Module Page
- Info on the **first 2 Books** we are going to use

Next Slide

Refreshing JavaScript (JS)

- This module is mainly **JavaScript-Based**
- You would have learnt **JavaScript** in the **2nd Year** (within **CST2120 Web Applications and Databases**):
 - (at least) the **basics** related to **Front-End** and **Back-End JavaScript** development
- If **JavaScript** is new to you or you need to **refresh** your memory, there are many **free books**.
 - I would recommend: **Eloquent JavaScript** available at the official [link](#)

ELOQUENT JAVASCRIPT

THIRD EDITION

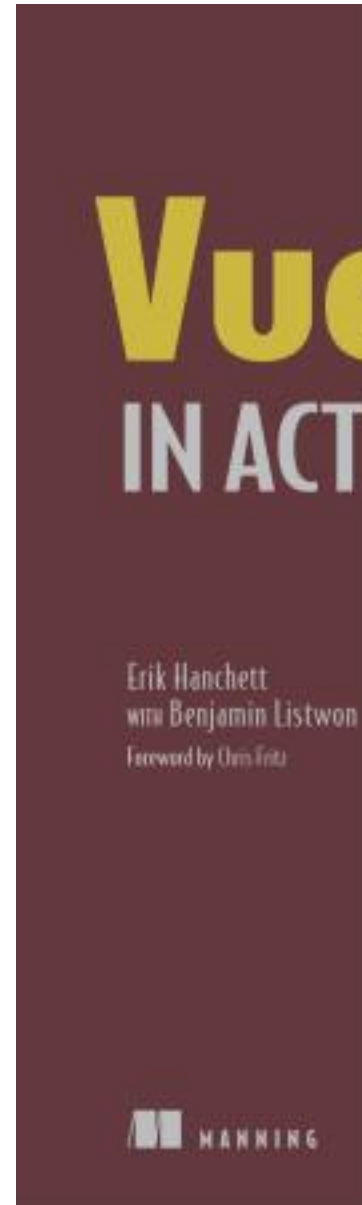
A Modern Introduction
to Programming

Marijn Haverbeke



Textbook

- Textbook: **Vue.js in Action**
 - **Vue.js Framework**
 - All the **code examples** are available at this GitHub repository: [link](#)



Integrated Development Environments (IDEs)

Why IDEs are important?

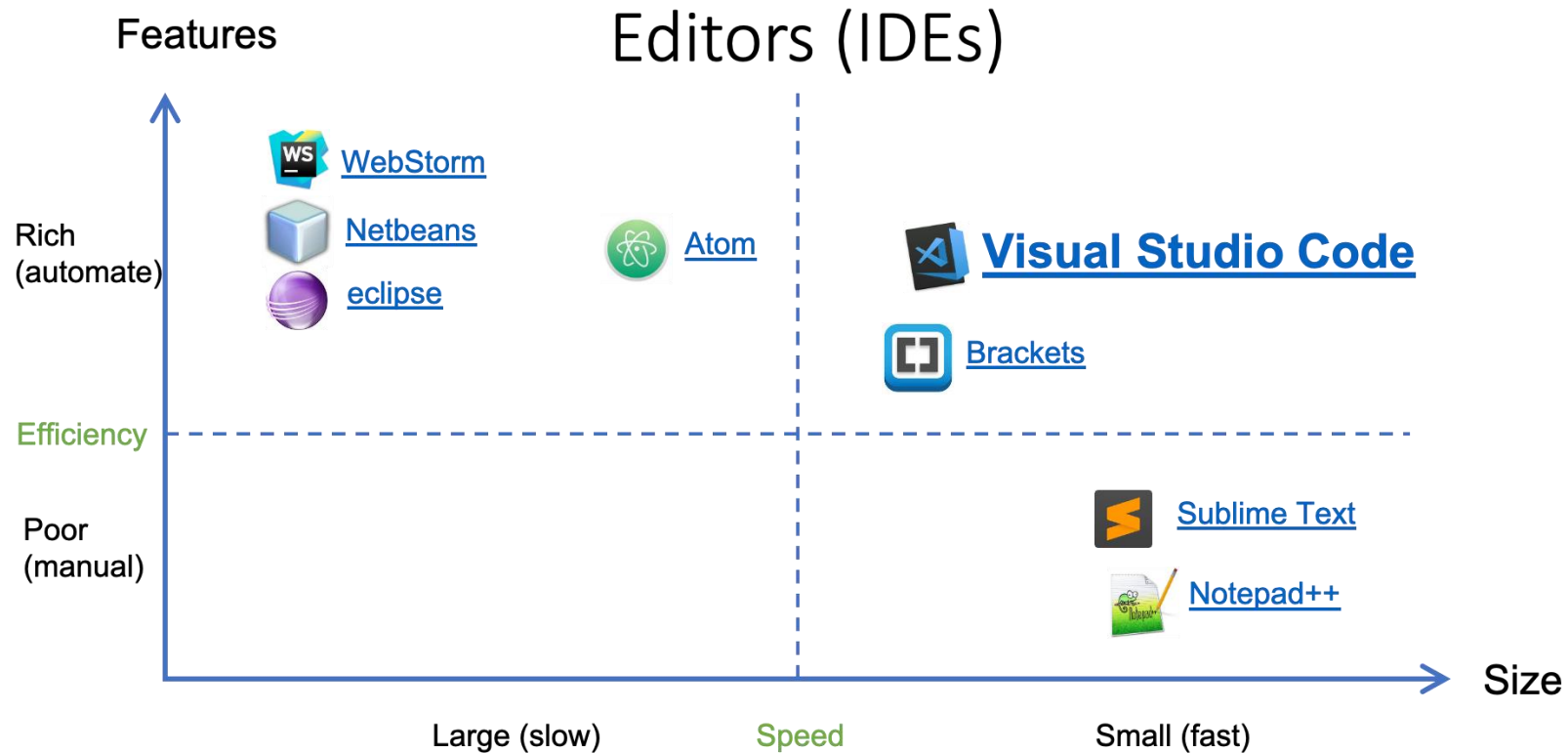
- to **speed up the coding**
 - auto completion
 - Emmet toolkit
- to **organise better the code** and **perform refactoring**
 - packages/folders structuring
 - merge and duplication identification
- to have **ready-to-use and powerful features**
 - code formatting
 - debugging
 - support for code versioning
- **visual aids**
- ...



In a sentence: to have everything you need for developing as a professional



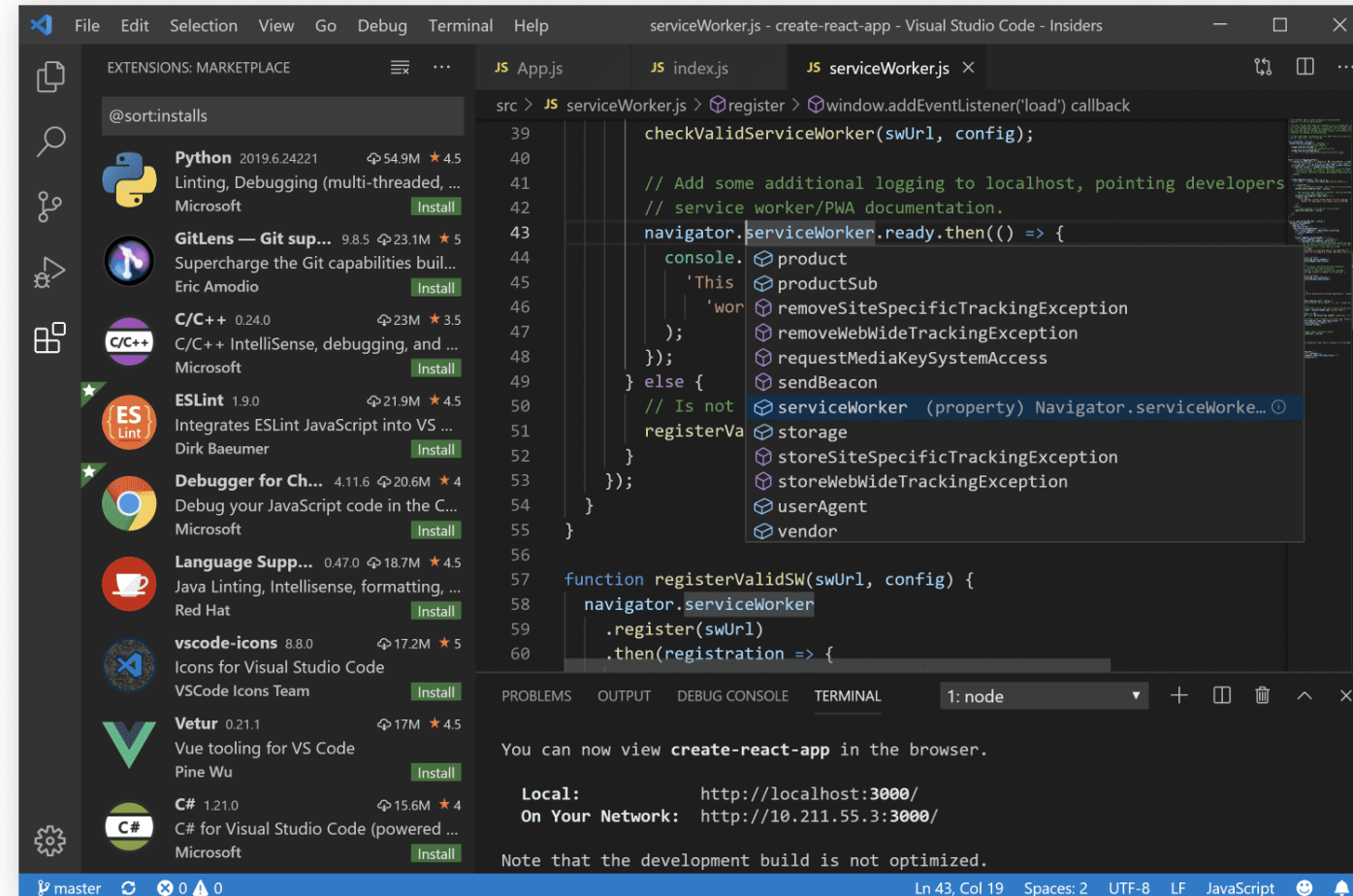
Available IDEs



What IDE are you familiar with?

Visual Studio Code (VSC) and Why this IDE?

- Visual Studio Code: [Official Website](#)
- Used by many professional developers
- Available on all operating systems: Windows, Mac, Linux/Unix
- Free and Open-Source
- Made by Microsoft, which also owns (since 2018) GitHub.com (we will use **GitHub** in this module as well)
- Good balance between features and size/performance
- Many available **Extensions**
- Probably **most of you have already used it** in previous modules
- We will use this one for this module, but **feel free to use the one you prefer**



Let's Start Using VSC: Emmet and its Documentation

- Emmet has its own official website (link: [Emmet](#))
- There you can find its official documentation (link: [docs](#))



Emmet Documentation

[← Back to main website](#)



Your new development career awaits. Check out the latest listings.

ads via Carbon

Abbreviations

Syntax
Element types
Implicit tag names
"Lorem Ipsum" generator

CSS Abbreviations

Abbreviations

Abbreviations are the heart of the Emmet toolkit: these special expressions are parsed in runtime and transformed into structured code block, HTML for example. The abbreviation's syntax looks like CSS selectors with a few extensions specific to code generation. So every web-developer already knows how to use it.

Here's an example: this abbreviation

```
#page>div.logo+ul#navigation>li*5>a{Item $}
```

...can be transformed into

```
<div id="page">
  <div class="logo"></div>
  <ul id="navigation">
    <li><a href="">Item 1</a></li>
    <li><a href="">Item 2</a></li>
    <li><a href="">Item 3</a></li>
    <li><a href="">Item 4</a></li>
    <li><a href="">Item 5</a></li>
  </ul>
</div>
```

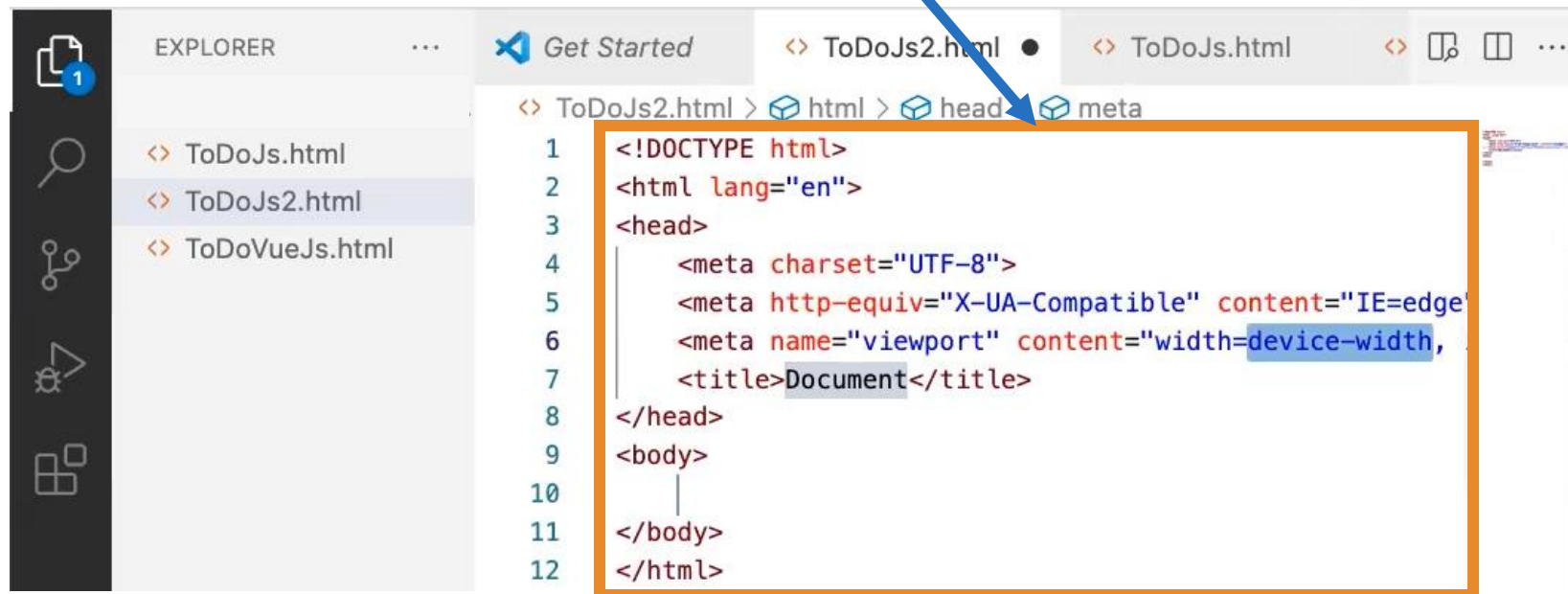
- Emmet is already integrated in VSC (examples in the next slides)

Next Slide

Let's Start Using Emmet within Visual Studio Code

Emmet is already integrated in VSC; in a html file let's try for instance:

- type `!` and press enter



Emmet and Visual Studio Code: other Examples (1)

Other Examples: Basic HTML elements

- type `input` and press enter

The image shows two screenshots of the Visual Studio Code editor demonstrating the use of Emmet abbreviations.

Top Screenshot: The Explorer sidebar shows files `ToDoJs.html`, `ToDoJs2.html`, and `ToDoVueJs.html`. The main editor shows the `ToDoJs.html` file with the following content:

```
1 <!--
2 <h1>
3 <h2>
4
5
6
7
8 </
9 <b
10 inp
11 </body>
12 </html>
```

A blue arrow points from the text "type `input` and press enter" to the `inp` abbreviation on line 10. A dropdown menu is open, showing a list of HTML input types: `input`, `input:b`, `input:btn`, `input:button`, `input:c`, `input:checkbox`, `input:color`, `input:date`, `input:datetime`, `input:datetime-local`, and `input:email`. The `input` option is highlighted. A blue arrow points from the `input` option to the "Emmet Abbreviation" box in the top right corner.

Bottom Screenshot: The Explorer sidebar shows files `ToDoJs.html`, `ToDoJs2.html`, and `ToDoVueJs.html`. The main editor shows the `ToDoJs2.html` file with the following content:

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta http-equiv="X-UA-Compatible" content="IE=edge">
6   <meta name="viewport" content="width=device-width, ...>
7   <title>Document</title>
8 </head>
9 <body>
10 |
11 </body>
12 </html>
```

A blue arrow points from the text "type `input` and press enter" to the `input` abbreviation on line 10. A dropdown menu is open, showing a list of HTML input types: `input`, `input:b`, `input:btn`, `input:button`, `input:c`, `input:checkbox`, `input:color`, `input:date`, `input:datetime`, `input:datetime-local`, and `input:email`. The `input` option is highlighted. A blue arrow points from the `input` option to the "Emmet Abbreviation" box in the top right corner.

Results: The final result of the expansion is shown in the bottom right corner, where the `input` abbreviation has been expanded into the following HTML element:

```
<input type="text" name="" id="">
```

Emmet and Visual Studio Code: other Examples (2)

Other Examples:

- `input+button+ul>li*3`
and press enter

- Now, try following and then press enter:

```
div#App>h1{My Todo  
List}+input#newTask+but  
ton#addButton{add}+ul#t  
asklist>li{Task}*3
```

Results

```
<body>  
  <input type="text"><button></button>  
  <ul>  
    <li></li>  
    <li></li>  
    <li></li>  
  </ul>  
</body>
```

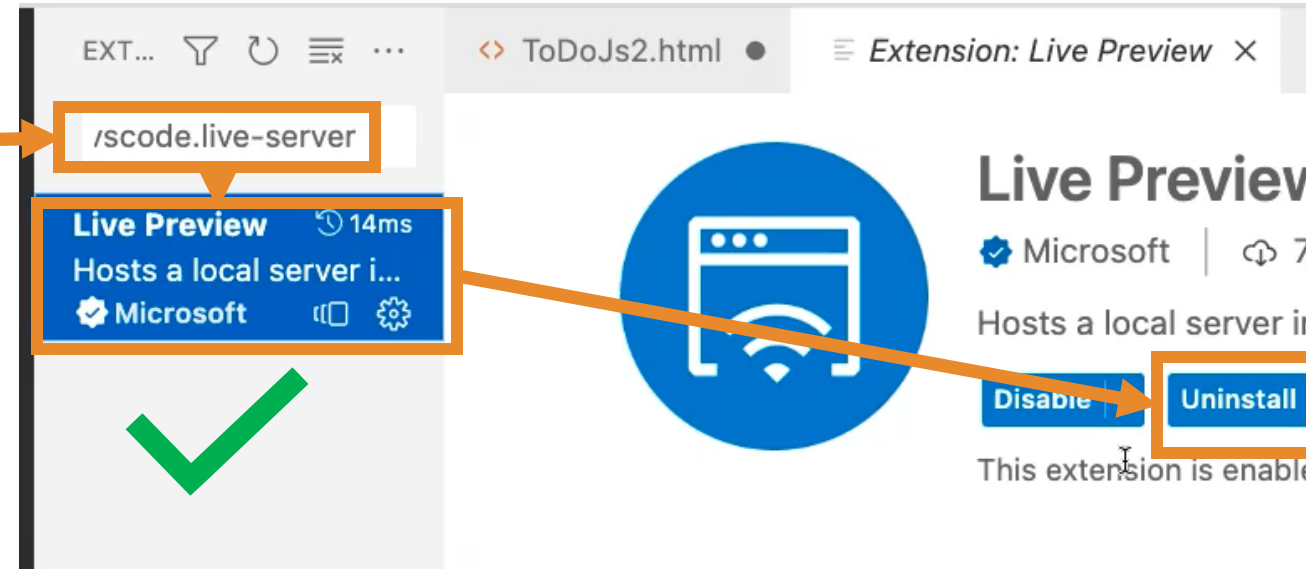
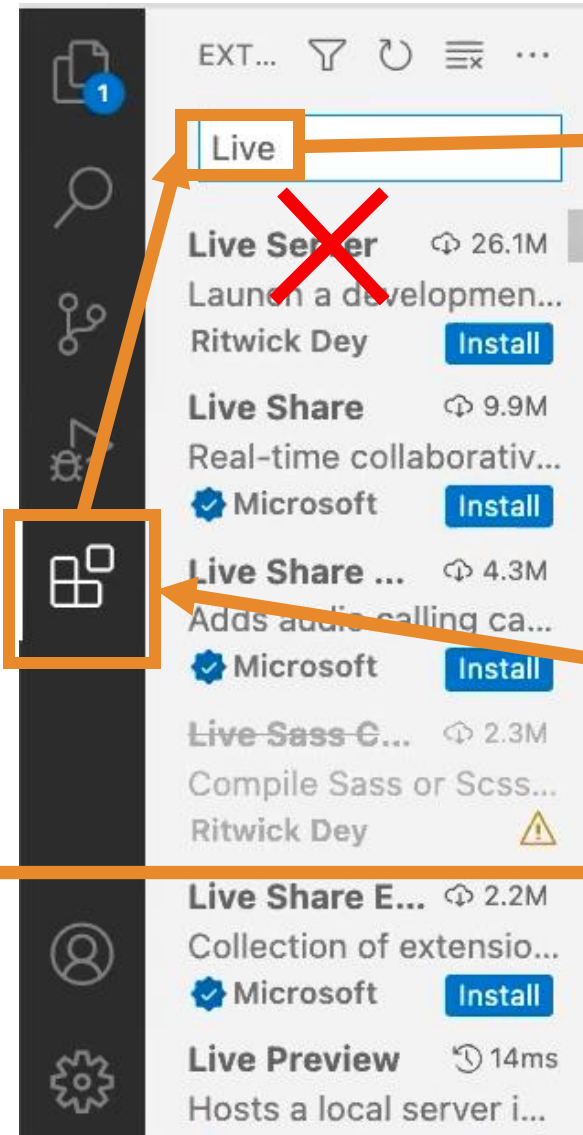
Results

```
<body>  
  <div id="App">  
    <h1>My TodoList</h1>  
    <input type="text" id="newTask"><button id="add">  
    <ul id="tasklist">  
      <li>Task</li>  
      <li>Task</li>  
      <li>Task</li>  
    </ul>  
  </div>  
</body>  
</html>
```

Let's start using VSC Extensions: Live Preview (Extension)

Extensions for Visual Studio Code:

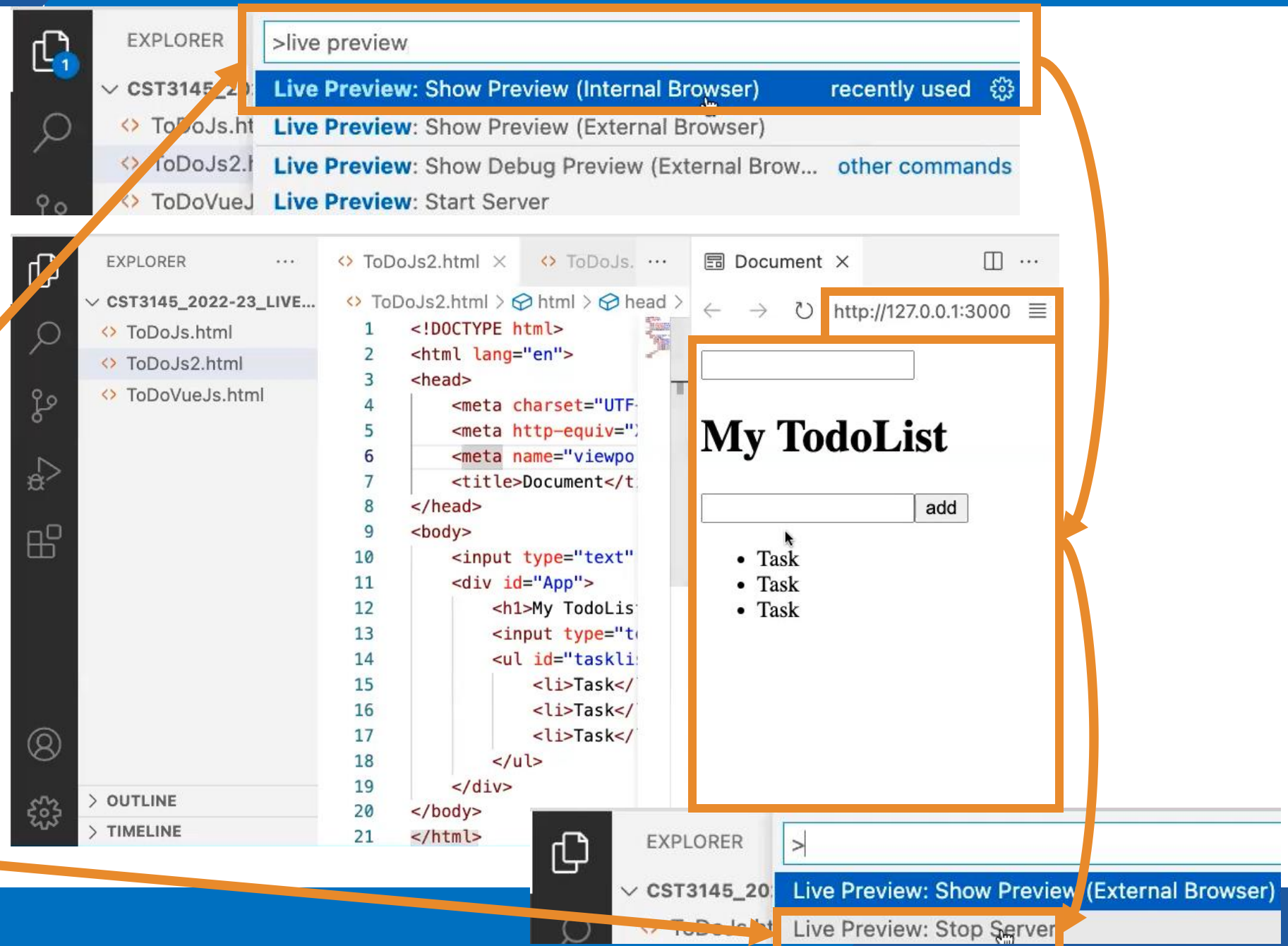
- **Live Server by Ritwick Dey:** (Extension ID: `ritwickdey.LiveServer`) this is **very good, mature but not lightweight**, so we do not use this, but the next one;



- **Live Preview by Microsoft:** (Extension ID: `ms-vscode.live-server`) this is **less mature, but lightweight**, we will use this a lot

Using Live-Preview and Command Palette of VSC

- **Command Palette:** from the top menu bar "View -> Command Palette" (or "ctrl shift p") depending on the system you are using (Mac, Linux, Windows);
- from command palette, if we write "live preview" we can start it;
- in the **extension settings of "live preview"** you can set if **to update what you are viewing continuously** or **only when saving changes**
- Remember to **stop the Server**



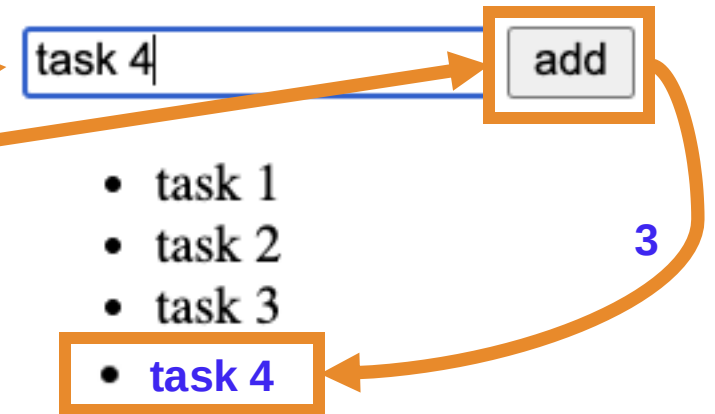
A JavaScript App: the “ToDo” App

Our “ToDo” App

- [Approach]

- We will use a **simple ToDo app** as an **example**:
 - type the new task in the text field;
 - press the “add” button and add it to the list.

My Todo List



- [Objectives]



- A quick review of JavaScript
- To compare the difference between plain JavaScript and Vue.js

Our “ToDo” App: the Code

My Todo List

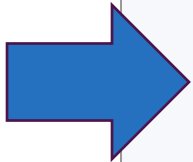
task 4

- task 1
- task 2
- task 3

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>todo</title>
</head>
<body>
  <h1>My Todo List</h1>
  <input type="text" id="newTask">
  <input type="button" value="add" id="addButton">
  <ul id="taskList"></ul>

  <script>
    let addButton = document.getElementById("addButton");
    let taskList = document.getElementById("taskList");
    addButton.onclick = function() {
      let taskField = document.getElementById("newTask");
      let taskItem = document.createElement("li");
      taskItem.innerText = taskField.value;
      taskList.appendChild(taskItem);
      taskField.value = "";
    }
  </script>
</body>
</html>
```

• The code of
our “ToDo”
App

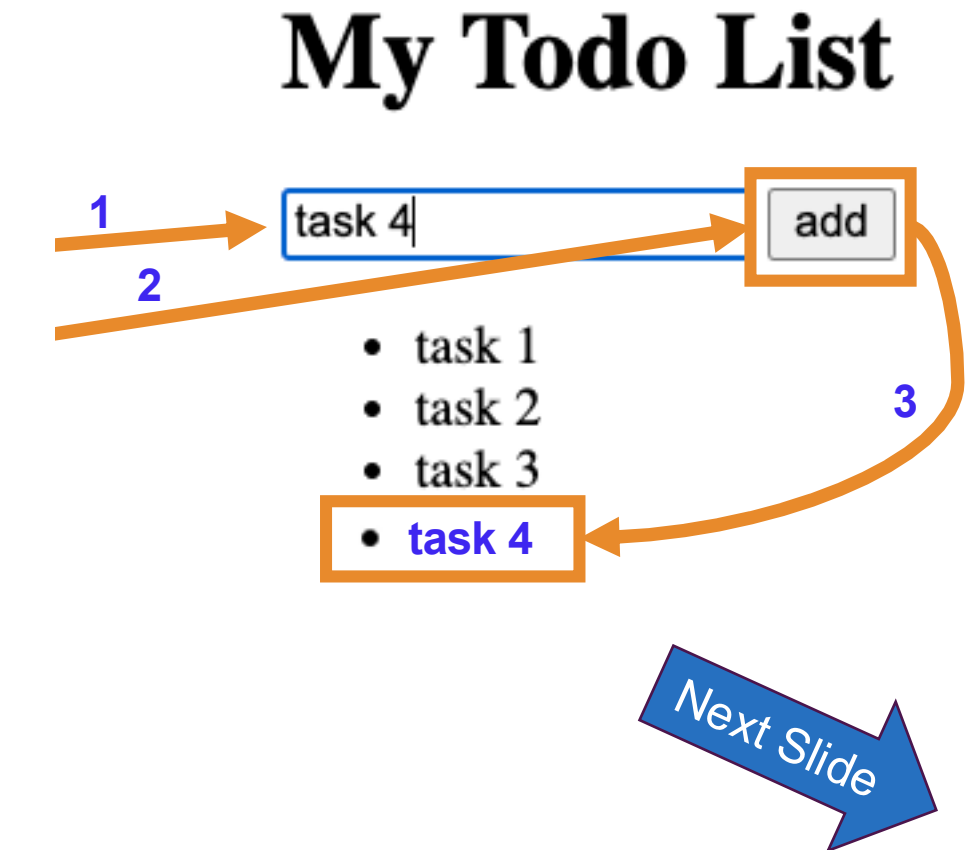


• “Pure”
JavaScript
version

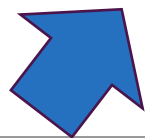
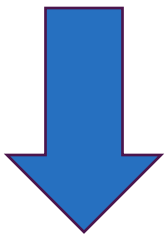
The `Vue.js` Version of Our “Todo” App

The `vue.js` Version

- The following solution has the **exact same function** but implemented with `vue.js`
- It is OK if some of the code does not make sense now; we will cover it fully in detail very soon.



- The code of our “ToDo” App
- Vue.js version



1

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <!--script src="https://unpkg.com/vue"></script-->
  <script src="https://unpkg.com/vue@2.7.8/dist/vue.js"></script>
</head>
<body>
  <div id="App">
    <h1>My TodoList</h1>
    <input type="text" id="newTask" v-model="newTask">
    <button id="addButton" v-on:click="addItem">add</button>
    <ul id="tasklist"></ul>
  </div>
```

```
<script>
  let app = new Vue({
    el: '#App',
    data: { newTask: '' },
    methods: {
      addItem: function() {
        let newItem = document.createElement('li');
        newItem.innerText = app.newTask;
        document.getElementById('tasklist').appendChild(newItem)
        app.newTask = '';
      }
    }
  })
</script>
</body>
</html>
```

2

My Todo List

- task 1
- task 2
- task 3

Comparison of JavaScript and Vue.js Versions

JavaScript vs Vue.js

- The two are **not that different**
 - Partly because **we use** `Vue.js` **as a JavaScript library.**
 - Later we will also try `Vue.js` **command line tools (CLI)**, which is more different.
- The code is **not necessarily shorter with** `Vue.js`
 - That is mostly because **this is a very simple app.**
 - For **more complex apps**, the powers of the `Vue Directives` will start to show.

A Slightly More Complex Example

- Calculator Example
 - On the left JavaScript Calculator: **124 lines**
 - On the right Vue.js Calculator: **45 lines**
- From 1st Chapter of our Textbook

```
78
79     var handleCalcEvent =
80         function handleCalcEvent(e) {
81             var newValues = lastValues,
82                 elem = e.target;
83
84             switch(elem) {
85                 case calc.xInput:
86                     newValues =
87                         lastValues.copyWith(
88                             Calc.xName,
89                             elem.value
90                         );
91                     break;
92                 case calc.yInput:
93                     newValues =
94                         lastValues.copyWith(
95                             Calc.yName,
96                             elem.value
97                         );
98                     break;
99             }
100
101             if(newValues !== lastValues){
102                 lastValues = newValues;
103                 lastValues.render();
104             }
105         };
106
107     elem.addEventListener('keyup', handleCalcEvent, false);
108     return lastValues;
109 }
110
111 window.addEventListener(
112     'load',
113     function() {
114         var cv = initCalc(document.getElementById('myCalc'));
115         cv.render();
116     },
117     false
118 );
119
120 }();
121 </script>
122 </body>
123 </html>
124
```

Listing 1.2. The Vue calculator: chapter-01/calculatorvue.html

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>A Vue.js Calculator</title>
5   <style>
6     p, input { font-family: monospace; }
7     p { white-space: pre; }
8   </style>
9 </head>
10 <body>
11   <div id="app">
12     <p>x <input v-model="x"></p>
13     <p>y <input v-model="y"></p>
14     <p>-----</p>
15     <p>= <span v-text="result"></span></p>
16   </div>
17
18   <script src="https://unpkg.com/vue/dist/vue.js"></script>
19   <script type="text/javascript">
20     function isNotNumericValue(value) {
21       return isNaN(value) || !isFinite(value);
22     }
23     var calc = new Vue({
24       el: '#app',
25       data: { x: 0, y: 0, lastResult: 0 },
26       computed: {
27         result: function() {
28           let x = parseFloat(this.x);
29           if(isNotNumericValue(x))
30             return this.lastResult;
31
32           let y = parseFloat(this.y);
33           if(isNotNumericValue(y))
34             return this.lastResult;
35
36           this.lastResult = x + y;
37
38           return this.lastResult;
39         }
40       }
41     });
42   </script>
43 </body>
44 </html>
45
```

Suggestions for Reading

Reading

- Vue.js in Action Book: Chapter 1

Questions?